

Nom et prénom :	Groupe :	Note :
-----------------	----------	--------

Question de cours :

2 Comment un sémaphore d'exclusion mutuelle garantit-il qu'un seul processus accède à une section critique à la fois ?

Exercice :

Soient trois processus P1, P2, P3 qui partagent les variables globales « n » et « m ». Pour contrôler l'accès à ces deux variables, on propose la solution suivante :

float n = 6 ;

int m = 6 ;

semaphore S1=1, S2 = 1;

```
void P1() {
  1 wait(S1);
  2 wait(S2);
  3 n = n / 3;
  4 m = m * 2;
  5 signal(S2);
  6 signal(S1);
}
```

```
void P2() {
  7 wait(S2);
  8 m = m + 3;
  9 signal(S2);
}
```

```
void P3() {
  10 wait(S1);
  11 n = n + 5;
  12 signal(S1);
}
```

10

- 3 1. Montrer que cette solution n'est pas correcte, et indiquer quelle est la condition qui n'est pas satisfaite parmi les 3 conditions requises pour réaliser une exclusion mutuelle correcte.
- 5 2. Proposer une solution correcte en utilisant les sémaphores qui synchronise les trois processus pour avoir toujours le résultat suivant : n = 7 et m = 18.

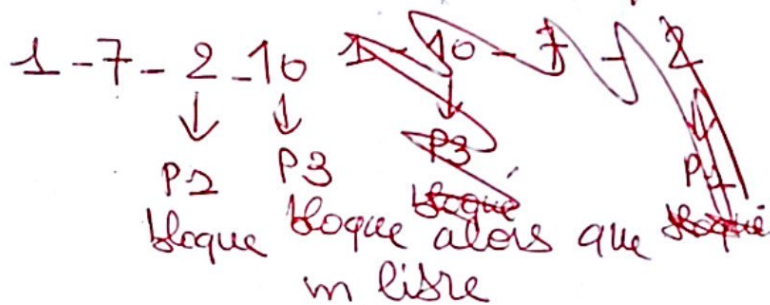
Réponse :

R1:

Progression non vérifiée

Justification :

P bloqué pour une SC libre



Nom et prénom :	Groupe :	Note :
-----------------	----------	--------

Question de cours :

Pourquoi les solutions basées sur les sémaphores sont-elles souvent préférées aux solutions basées sur l'attente active dans les systèmes d'exploitation ?

Exercice :

Soient deux de processus concurrents effectuant chacun une opération particulière : P1 effectue la tâche T1 et P2 effectue la tâche T2.

- Utiliser un sémaphore pour garantir toujours que l'exécution de la tâche T1 ne soit jamais synchronisée avec l'exécution de la tâche T2.
- Utiliser deux sémaphores pour synchroniser les deux processus de telle manière que l'ordre d'exécution des deux tâches T1 et T2 soit comme suit : T2 T1 T2 T1 T2 T1 T2

```
Void P1 () {
```

```
.....
```

```
While (1){
```

```
    T1
```

```
}
```

```
.....
```

```
}
```

```
Void P2 () {
```

```
.....
```

```
While (1){
```

```
    T2
```

```
}
```

```
.....
```

```
}
```

Réponse :

R₁ : semaphore s = 1,

P1() {

while(1) {

wait(s);

T1

signal(s);

}

}

P2() {

while(1) {

wait(s);

T2

signal(s);

}

}

Nom et prénom :	Groupe :	Note :
-----------------	----------	--------

Questions de cours :

1. Quelle est la différence entre un sémaphore d'exclusion mutuelle et un sémaphore général?
2. Que se passe-t-il si un processus exécute un wait sur un sémaphore dont la valeur est zéro ?

Exercice :

semaphore a=1, b=1, c=0 ;

P1	P2
.....	
1) wait (a)	6) wait (a)
2) signal (a)	7) wait (b)
3) wait (b)	8) wait (c)
4) signal (b)	9) signal (a)
5) signal (c)	10) wait (b)
.....	

1. Est-ce que les deux processus P1 et P2 génèrent un interblocage ? Justifier votre réponse.
2. Choisir l'affichage (ou les affichages) possible à l'exécution des trois processus suivants :

Semaphore a=0, b=1 ;

P1 () { while(1){ wait (a); A1; signal (b); } }	P2 () { while(1){ wait (b); A2; signal (a); } }	P3 () { while(1){ wait (a); A3; signal (b); } }
---	---	---

- 1) A3 A2 A2 A1
- 2) A2 A3 A2 A1
- 3) A2 A3 A1 A2
- 4) A2 A1 A3 A1
- 5) A2 A1 A2 A1

oui Interblocage: 1 - 2 - 6 - 7 - 8 - 3

6 - 7 - 1 - 8 -

6 - 1 - 7 - 8

6 - 7 - 8 - 1

1 - 6 - 2 - 7 - 8 - 3

Nom et prénom :	Groupe :	Note :
-----------------	----------	--------

Questions de cours :

Supposons que la tâche A s'exécute avant la tâche B.

- 2
1. À quel moment une tâche doit-elle exécuter un « wait » pour respecter l'ordre d'exécution ?
 2. Quand une tâche doit-elle exécuter un « signal » pour permettre à une autre tâche de commencer son exécution ?

Exercice :

1) Considérons deux processus dont le code est donné ci-dessous :

semaphore a=1, b=0, c=1 ; /* déclarations globales */

a' - b' - a - b -
c' - c - d - b
3

P1	P2
..... a) wait(a) b) signal(b) c) wait(b) d) wait(c) e) signal(a)	 a') wait(a) b') signal(a) c') wait(c) d') wait(b) e') signal(b) f') signal(c)

Est-ce que ces processus risquent de générer un interblocage ? Justifier votre réponse.

2) Soient trois processus P1, P2, et P3 définis ci-dessous :

semaphore a=0, b=1;

5

P1 () { while(1){ wait(a) A1; signal(b); } }	P2 () { while(1){ wait(b); A2; signal(a); } }	P3 () { while(1){ wait(a); A3; signal(b); } }
--	---	---

Compléter le code ci-dessus pour remplir les conditions suivantes :

- Lors de la première itération, A2 doit être exécutée en premier.
- A chaque itération, les processus doivent être synchronisés afin de respecter l'ordre d'exécution suivant : A2-A1-A2-A3-A2-A3-A2-A3-A2-A1-.....
- Exemple des séquences interdites : A2-A1-A3-A2-A1-A2-A3..... ; A3-A1-A2-A2-A1-....
A2-A3-A1-A2-.....

Test (30min)

Nom et prénom :	Groupe :	Note :
-----------------	----------	--------

Exercice 1:

Soit l'expression arithmétique suivante : $(a + b) * c - d / (e - f)$. On suppose que chacune des opérations de l'expression est réalisée par un processus à part. Proposer une solution (pseudo code) qui synchronise les processus pour calculer correctement l'expression arithmétique en utilisant des sémaphores.

Exercice 2:

Une piscine qui possède N paniers disponibles à l'entrée et M cabines d'habillage ($1 \leq M \ll N$) peut accueillir N nageurs au max. Un nageur qui arrive doit attendre s'il n'y a pas de paniers disponibles. Lorsqu'il acquiert un panier, il doit occuper une cabine d'habillage (si disponible) pour se changer et aller nager. Dès qu'il termine à nager, il occupe encore une fois une cabine pour se rhabiller et libérer son panier à la sortie. Ecrire l'algorithme du processus **Nageur** en utilisant les deux sémaphores « panier » et « cabine ».

```
Void Nageur ( ) {  
    < mettre_habits_dans_panier >  
  
    < nager >  
  
    < se_rhabiller >  
}
```

Réponse :

Nom et prénom :	Groupe :	Note :
-----------------	----------	--------

Questions de cours :

Supposons que la tâche A s'exécute avant la tâche B.

1. À quel moment une tâche doit-elle exécuter un « wait » pour respecter l'ordre d'exécution ?
2. Quand une tâche doit-elle exécuter un « signal » pour permettre à une autre tâche de commencer son exécution ?

Exercice :

Soit un tableau T de 10 éléments de type entier. Pour calculer la somme des éléments de ce tableau on utilise une seule valeur partagée « somme » et deux processus parallèles, P1 pour parcourir les éléments d'indice pair, et P2 pour parcourir les éléments d'indice impair. Donner les algorithmes de P1 et P2 en utilisant un sémaphore pour garantir que le résultat soit toujours correct.

int T[10]; // Déclaration globale d'un tableau des 10 entiers.

int somme = 0; // Déclaration globale de la valeur somme des éléments du tableau T.

void P1() { *pair*
.....
}

void P2() { *impair*
.....
}

void main () {
// Remplissage du tableau//
Parbegin (P1(), P2());
// Affichage du résultat de la
valeur somme //
}

Réponse: *sémaphore mutex = 1;*

<pre> 1 P1() { for int i = 0; while while (i <= 9) { 1 wait(mutex); en somme = somme + T[i]; 1 signal(mutex); en i = i + 2; } } </pre>	<pre> P2() { 0,21 int j = 1; 0,21 while (j <= 10) { 1 wait(mutex); en somme = somme + T[j]; 0,1 signal(mutex); en j = j + 2; } } </pre>
---	---

R2;

semaphore $S1 = 1, S2 = 0;$

P1() {

while(1) {

wait(S1);

T1

signal(S2);

}

}

P2() {

while(1) {

wait(S2);

T2

signal(S1);

}

}

P1() {

while(1) {

wait(S2);

T1

signal(S1);

}

}

P2() {

while(1) {

wait(S1);

T2

signal(S2);

}

}

T1 T2 T1 T2 ...

T2 T1 T2 ...

Q2.

int n=6 ; float m=6;
semaphore S1=0, S2=0;

P1	P2	P3
m = m/3; signal(S1); wait(S2); n = n * 2;	n = n + 3; signal(S2);	wait(S1); m = m + 5;

S1=1, S2=3

n = 18
m = 7

int n=6 ; float m=6;
semaphore S1=0, S2=0;

P1	P2	P3
n = n / 3; signal(S1); wait(S2); m = m * 2;	m = m + 3; signal(S2);	wait(S1); n = n + 5;

S1=1, S2=1

2